

AI-SHA Foundation Tutorial

Volume II — CP5, CP6 and the Skill Layer: giving the brain eyes, ears and jobs

Volume I took a blank Jetson to a built workspace (FLASH through CP3). This volume covers what the reference build did next and proved on real hardware: the camera and detection stack (CP5), the voice and knowledge loop (CP6), and two working **skills** — administrator face authentication and video messaging. Same rules as Volume I: every command here already dodges something that cost us hours, and the coloured boxes tell you what would otherwise have gone wrong. Read the gate at the end of each section before moving on.

Where you are



The traps this tutorial defuses for you

TRAP AVOIDED

Ten detours removed from your path. (1) **Install the Jetson-specific torch wheel** — the generic PyPI torch runs CPU-only on this board and you will not notice until inference crawls. (2) **Install ultralytics with `--no-deps`** — otherwise it drags in an OpenCV that shadows the JetPack `cv2`. (3) **Pin `protobuf==4.25.8`** or mediapipe and the ML stack fight. (4) **Use the mediapipe Tasks API**, not `mediapipe.solutions` from-imports. (5) **Kill `yoLov8_node` explicitly** — the arbiter spawns it detached and it survives the launch, holding the GPU. (6) **Address the microphone by name (`ReSpeaker`)**, never `plughw:1,0` — that is the dead on-board APE. (7) **Retry the audio open** — the array throws ALSA `-9985` on a release race. (8) **Use `Llama3.2:1b`**, never the bare tag — it resolves to `:latest` and 404s. (9) **Never trust liveness that only measures depth range** — a printed photo defeats it; you need planarity. (10) **Install the JetPack build of Ollama and cap its context** — otherwise the LLM silently runs on the CPU and answers take 30-80 s instead of 2-4 s.

FROM THE FIELD

Version anchor. Everything here was validated on the ISC-Sharjah reference build: JetPack 6.2.2 (L4T R36.5.0), ROS 2 Humble, master at `c622edf` plus the CP5/CP6 work and the two skill branches. The Jetson in the reference build is user `hsdaou` on host `orin-nano`, reached over the USB device-mode link at `192.168.55.1` — substitute your own.

DECISION — WHY CP4 IS SKIPPED HERE

CP4 is the DDS mesh: the first time the Jetson and the Pi 5 talk to each other. It is **not covered in this volume because it was not achieved on the reference build** — the Pi 5 is still blocked on hardware (see the closing page). Everything in CP5, CP6 and the skills runs on the **Jetson alone** and was validated that way, so you can follow this volume before your Pi exists. The one visible consequence: the robot **cannot speak yet**. It hears, reasons and answers, but the answer is published as text on `/robot_speech` and waits for the Pi's text-to-speech at CP4.

The RealSense D435 gives colour and depth; YOLOv8 finds objects; mediapipe finds faces and hands. The traps here are all about **which build of which library** you install — the wrong torch and the wrong OpenCV both install perfectly happily and then quietly cripple the board.

1 Confirm the D435 is on a USB 3 port

Depth streaming needs SuperSpeed. On USB 2 the camera enumerates, the driver starts, and frames simply never arrive at the advertised rate:

JETSON

```
# must report 5000M - 480M means USB 2 and depth will starve
lsusb -t | grep -i -B1 realsense
rs-enumerate-devices | head -20      # serial + firmware appear
```

2 Install the Jetson build of torch (not the PyPI one)

This is the single highest-value line in CP5. The generic wheel is compiled against a CUDA that does not match JetPack, so it silently falls back to CPU:

JETSON

```
pip install --no-cache-dir \
  --index-url https://pypi.jetson-ai-lab.io/jp6/cu126 \
  torch==2.8.0 torchvision==0.23.0
python3 -c "import torch; print(torch.__version__, torch.cuda.is_available())"
# MUST print: 2.8.0 True      - if False, you have the wrong wheel
```

TRAP AVOIDED

What bit us: `pip install torch` pulled a generic `2.12.x+cu130` build. It imports, it runs, `torch.cuda.is_available()` returns `False`, and every model silently runs on the CPU — so you diagnose a "slow model" for hours instead of a wrong wheel. Check the boolean, not the version string.

3 Install ultralytics WITHOUT its dependencies

Ultralytics depends on its own `opencv-python`. Installed normally it shadows the JetPack-built `cv2` 4.5.4 that every other node already uses:

JETSON

```
pip install --no-deps ultralytics==8.4.89
python3 -c "import cv2, ultralytics; print(cv2.__version__, ultralytics.__version__)"
# expect: 4.5.4 8.4.89      - a 4.10/4.11 cv2 means the shadow happened
```

4 Pin protobuf for mediapipe compatibility

JETSON

```
pip install "protobuf==4.25.8"
# add the same pin to constraints-jetson.txt so it survives the next install
```

Note: Volume I's inline pin list omits this one. The committed `constraints-jetson.txt` in the reference build has it — if you are typing the pins by hand, add `protobuf==4.25.8`.

5 Fetch the mediapipe Tasks models

The models are binary assets, deliberately kept out of git. The repo ships a fetch script:

JETSON

```
cd ~/robot_ws && ./scripts/fetch_mediapipe_task_models.sh
ls -la models/
# expect: blaze_face_short_range.tflite  hand_landmarker.task
```

TRAP AVOIDED

Why the code uses the Tasks API. On this mediapipe build, `from mediapipe.solutions import face_detection` raises `ImportError`, while `mp.solutions.face_detection` works — the package resolves submodules lazily through `__getattr__`. That inconsistency makes the legacy API a trap for anyone editing the nodes later, so the reference build was ported to the **Tasks API** with the `.tflite` / `.task` files above. Do not "fix" a from-import back in.

6 Build the TensorRT engine (one-off, ~15 min)

PyTorch weights run; a TensorRT engine runs about three times faster. Build it once on the board it will run on — engines are not portable between machines or JetPack versions:

JETSON

```
cd ~/robot_ws
yolo export model=yolov8m.pt format=engine device=0 half=True
ls -lh yolov8m.engine      # ~52 MB on the reference build
```

Note: the exporter auto-installs `onnx`, `onnxruntime`, `onnxslim`, `coloredlogs` and `humanfriendly` as it runs. That is expected and did not disturb the CP1 pins — but do the export **after** the constraints are in place, not before.

7 Launch and confirm detections on real frames

JETSON

```
source /opt/ros/humble/setup.bash && source ~/robot_ws/install/setup.bash
export ROS_DOMAIN_ID=99
ros2 launch realsense2_camera rs_launch.py align_depth.enable:=true &

ros2 topic hz /camera/camera/color/image_raw          # ~30 Hz
ros2 topic hz /camera/camera/aligned_depth_to_color/image_raw # ~27 Hz
ros2 topic echo /detection/faces_simple --once        # conf + depth in metres
```

FROM THE FIELD

Measured on the reference build. Colour 30 Hz, aligned depth 27 Hz, D435 negotiated at 5000M. YOLOv8m through TensorRT: ~35 ms per frame (~28 FPS). A face detected at confidence 0.86 and 0.42 m on `/detection/faces_simple`. If your depth rate is roughly half the colour rate, that is normal; if it is zero, revisit step 1.

TRAP AVOIDED

The orphan that holds your GPU. The arbiter starts `yolov8_node` as a *detached* `ros2 run` child. Killing the launch leaves it alive, still holding the GPU and the camera — so your next launch fails on a busy device and the cause is invisible. Always shut down with an explicit kill, and use a bracketed pattern so the command does not match its own shell:

JETSON

```
pkill -9 -f 'yolov8_nod[e] '
pkill -9 -f 'realsense2_camera_nod[e] '
```

GATE - proceed only when:

- ☐ `torch.cuda.is_available()` prints **True** and `cv2.__version__` is still 4.5.4.
- ☐ `yolov8m.engine` exists and the node reports frames in the 25-30 FPS range.
- ☐ Colour ~30 Hz and aligned depth ~27 Hz on the two camera topics.
- ☐ A real face appears on `/detection/faces_simple` with a plausible depth in metres.
- ☐ After shutdown, `pgrep -f 'yolov8_nod[e]'` returns nothing.

CP6

Voice and knowledge - giving it ears

Runs on: Jetson Orin Nano (Jetson half of CP6) Time: ~0.5 day

The loop is: microphone → Whisper transcription → wake word → intent → RAG retrieval over the school knowledge base → local LLM answer → published as text. On the Jetson alone the answer arrives as a ROS message; the spoken half waits for the Pi 5.

1 Find the microphone and prove it captures

JETSON

```
arecord -l | grep -i respeaker
# card 2: ArrayUAC10 [ReSpeaker 4 Mic Array (UAC1.0)], device 0
arecord -D plughw:ArrayUAC10,0 -f S16_LE -r 16000 -c 6 -d 3 /tmp/t.wav
python3 -c "import wave, numpy as np; w=wave.open('/tmp/t.wav'); \
    d=np.frombuffer(w.readframes(w.getnframes()), '<i2'); \
    print('RMS', int(np.sqrt((d.astype(float)**2).mean())))"
# RMS in the hundreds = speech present. Single digits = wrong device.
```

TRAP AVOIDED

Three names, only one works. The default `plughw:1,0` is the Jetson's on-board APE — it is a real ALSA card that captures pure silence, so you get a working pipeline with no audio and nothing to debug. `plughw:2,0` is the right hardware but the launch file's device matcher does not recognise the numeric form. **Pass the name:** `audio_device:=ReSpeaker`.

2 Pre-fetch the Whisper model while you still have bandwidth

JETSON

```
python3 -c "from faster_whisper import WhisperModel; \
    WhisperModel('base', device='cpu', compute_type='int8')"
# downloads + caches; do this before you need the robot to work
```

Note: STT runs on the **CPU**. There is no aarch64 CUDA build of CTranslate2 for this platform, so GPU transcription is not available — `base` at `int8` runs about 1.3× realtime, which is fine for turn-taking. Do not lose a day trying to force it onto the GPU.

3 Launch the voice stack

JETSON

```
ros2 launch aisha_bringup jetson_launch.py \
    audio_device:=ReSpeaker llm_model:=llama3.2:1b
```

TRAP AVOIDED

The bare model tag 404s. `llm_model:=llama3.2` resolves to `llama3.2:latest`, which is *not* what CP1 pulled — so the admin node gets a 404 from Ollama on the first question and the robot goes mute with no obvious cause. Always the explicit `:1b` tag. The reference build changed the launch default so a fresh follow-along cannot hit this.

TRAP AVOIDED

ALSA -9985, "Device unavailable". The ReSpeaker is a UAC1.0 device and does not release its capture endpoint instantly — restart a node too quickly and the open fails. The fix is not a longer sleep, it is a retry: **4 attempts, 3 seconds apart**. Both `stt_node` and the video-message skill do this; anything new you write against the array must too.

4 Say the wake word, watch the transcription

JETSON

```
ros2 topic echo /speech/text
# say: "Hey Aisha, can you hear me?"
# expect wake_word_triggered in the node log, then the stripped question
```

Note: the wake regex is deliberately punctuation-tolerant — Whisper writes *"Hey, Aisha!"* with a comma, which a naive `"hey aisha"` in text check misses. It also accepts the *Ayesha* and *Aysha* spellings, because the transcriber picks whichever it likes on the day.

5 Verify the knowledge loop WITHOUT a microphone

This is the most useful debugging technique in CP6: skip audio entirely and inject the text a perfect transcription would have produced. It separates "the microphone is wrong" from "the knowledge base is wrong", which otherwise look identical:

JETSON

```
ros2 launch aisha_bringup jetson_launch.py enable_stt:=false &
# WAIT for "Knowledge Base Online" in the log before injecting
ros2 topic pub --once /speech/text std_msgs/String \
  "{data: 'what are the tuition fees?'}"
ros2 topic echo /robot_speech
```

TRAP AVOIDED

The cold-start race. Publish before the knowledge base finishes loading and your message lands on a topic nobody is subscribed to yet — it is silently discarded, you see no answer, and you conclude RAG is broken. Wait for **"Knowledge Base Online"**. This costs one second and saves an afternoon.

6 Confirm the arbiter frees the GPU for the LLM

Vision and language cannot both own the GPU on an 8 GB board. The arbiter flips between `NAVIGATING` and `CONVERSING`; on the flip the admin node re-offloads the LLM and the vision node is stopped:

JETSON

```
ros2 topic pub --once /speech/text std_msgs/String "{data: 'aisha stop'}"
# log: mode=CONVERSING -> LLM num_gpu=999 (and yolov8_node terminated)
# the reverse flip logs: mode=NAVIGATING -> LLM num_gpu=0
```

FROM THE FIELD

What a passing CP6 looked like. Wake word triggered on a spoken *"Hey, Aisha!"*; a spoken tuition question returned a grounded answer citing the real fee table (Grade 7, AED 31,500) from 5 retrieved chunks of the 621-chunk knowledge base, generated by `llama3.2:1b`; and the `aisha stop` command flipped the arbiter, freed the GPU and re-offloaded the LLM. The answer was **text on a topic** — nothing was audible, and that is correct until the Pi arrives.

A freshly built AI-SHA answers correctly but takes **30 to 80 seconds**, which is unusable in a corridor. It is not that the model is big — `llama3.2:1b` is tiny. It is that **four separate things must all be right** before the Orin's GPU is actually used, and every one of them fails silently. On the reference build all four were wrong at once. Fix them and the same question answers in **2-4 seconds**.

1 First, check whether the LLM is on the GPU at all

One command decides everything that follows:

JETSON

```
ollama ps
# PROCESSOR column: 100% GPU = healthy    100% CPU = you have this problem
```

TRAP AVOIDED - THE BIG ONE

Ollama ignores the Orin's GPU by default. The Orin's GPU is **integrated** — it shares the 8 GB system RAM instead of having its own VRAM — and Ollama skips integrated GPUs unless told otherwise. It reported `total_vram="0 B"` and ran everything on the CPU. On top of that the generic arm64 build carries no Tegra CUDA runner at all. **You need the JetPack build**, which ships a `cuda_jetpack6` runner built against CUDA 12.6.

JETSON

```
# match YOUR installed version - and note the extension is .tar.zst, NOT .tgz
V=$(ollama --version | grep -oE '[0-9]+\.[0-9]+\.[0-9]+')
wget --continue https://github.com/ollama/ollama/releases/download/v$V/ollama-linux-arm64-jetpack6.tar.zst
sudo cp -a /usr/local/lib/ollama /usr/local/lib/ollama.bak      # rollback safety
sudo systemctl stop ollama
zstd -d ollama-linux-arm64-jetpack6.tar.zst -o /tmp/o.tar && sudo tar -C /usr/local -xf /tmp/o.tar
sudo systemctl start ollama

# proof it worked:
journalctl -u ollama | grep "inference compute"
# library=CUDA compute=8.7 name=Orin type=iGPU total="7.4 GiB"  <- correct
```

2 Enable the integrated GPU; keep the model resident

JETSON

```
sudo mkdir -p /etc/systemd/system/ollama.service.d
sudo tee /etc/systemd/system/ollama.service.d/override.conf <<'EOF'
[Service]
Environment="OLLAMA_IGPU_ENABLE=1"
Environment="OLLAMA_KEEP_ALIVE=30m"
EOF
sudo systemctl daemon-reload && sudo systemctl restart ollama
```

Note: `admin_node` also sends `keep_alive` on every request, and a per-request value **overrides** the server default. It shipped as `"30s"`, so the 1.5 GB model was evicted between questions and each query paid a full GPU reload. Change it to `"30m"` in `admin_node.py` as well, or the setting above achieves nothing.

3 Cap the context, or CUDA runs out of memory

With the GPU finally in use, the robot began failing outright with *"I encountered an error processing your question."* The cause is specific to a shared-memory board:

TRAP AVOIDED

A 131k context does not fit next to ROS. llama3.2 advertises a **131 072**-token context. Sized for that, the **KV cache alone requests ~2 GiB** — and on the Orin that is the same 8 GB the camera driver, Whisper and ChromaDB are already using. You get `cudaMalloc failed: out of memory`, llama-server **segfaults**, and the robot returns a generic error that tells you nothing about the real cause. It only ever feeds the model ~5 knowledge chunks, so a 2048 context is generous.

Bake the small context into a dedicated model so every caller inherits it:

JETSON

```
cat > Modelfile <<'EOF'
FROM llama3.2:1b
PARAMETER num_ctx 2048
EOF
ollama create aisha:1b -f Modelfile
# then launch with llm_model:=aisha:1b
ollama ps      # CONTEXT must read 2048, PROCESSOR 100% GPU
```

4 Fix the client that overrides your context anyway

TRAP AVOIDED - WHY THE MODELFILE ALONE IS NOT ENOUGH

Even with the Modelfile in place the server still logged `n_ctx = 131072`. The culprit is the client: llama_index's **ollama** defaults **context_window** to **-1**, meaning "use the model's maximum", and sends *that* as the request's `num_ctx` — overriding both the Modelfile and the server environment. Passing `num_ctx=2048` is **not** sufficient; `context_window` must be set too, in `admin_node.py`:

JETSON

```
self._llm_kwargs = dict(
    model=llm_model, base_url=ollama_url,
    request_timeout=llm_timeout, keep_alive="30m",
    num_ctx=ollama_num_ctx,
    context_window=ollama_num_ctx,  # <- without this, 131072 wins
)
```

FROM THE FIELD

Measured on the reference build. Before: **30-80 s** per answer, `ollama ps` reading 100% CPU, the GPU idle on a board chosen for its GPU. After all four fixes: **2 s, 3 s and 4 s** on three consecutive school questions, **100% GPU**, context 2048, model resident — roughly **20x faster** with the same model, knowledge base and answers.

Diagnose in this order — each symptom masks the next: `ollama ps` → `journalctl -u ollama | grep "inference compute"` → `grep n_ctx` → `keep_alive`.

DECISION — WHO GETS THE GPU, AND WHEN

Vision (YOLO holds ~1.2 GB of CUDA context) and the LLM cannot comfortably share an 8 GB board, so the natural design is to hand the GPU back and forth: load the LLM while conversing, return it to YOLO afterwards. **The repo already implements exactly this** — `gpu_arbiter` publishes `/aisha/mode`, and `admin_node` responds with `num_gpu=999` when **CONVERSING** and `num_gpu=0` when **NAVIGATING**, terminating `yolo_v8_node` on the flip. **Beware the default:** `admin_node` starts in **NAVIGATING**, so if you run *without* the arbiter (as the browser console does) nobody ever publishes a mode and the LLM stays pinned to the CPU forever. Either run the arbiter, or publish **CONVERSING** on `/aisha/mode` yourself with transient-local (latched) durability.

GATE - proceed only when:

- ☐ A 3-second capture from the ReSpeaker has an RMS in the hundreds, not single digits.
- ☐ Speaking the wake word puts your stripped question on `/speech/text`.
- ☐ An injected school question returns a grounded answer on `/robot_speech` — one that quotes real figures from the knowledge base, not a generic reply.
- ☐ The arbiter flip appears in the log with `num_gpu` changing, and no orphaned vision node is left behind.

DECISION — DNS, THE RECURRING SABOTEUR

On a school network the upstream resolver address changes without warning, and the symptom is never "DNS is broken" — it is `build_ros.sh` hanging on its `setuptools` pin, or `pip` timing out mid-install. When anything network-shaped misbehaves, re-point the resolver first: read the host's current one with `resolvectl status | grep 'Current DNS'` and write it to `/etc/resolvconf/resolv.conf.d/head` on the Jetson.

The skill layer

With eyes and ears working, the robot can be given **jobs**. A skill in this build is a small, self-contained program that uses the sensors already running, adds **no new pip dependencies** (the pinned stack from CP1 is fragile and must not be disturbed), keeps its data **outside the repository**, and ships with a gate that says when it is working. The two below are complete and were validated on real hardware.

SKILL 1

Administrator face authentication

Runs on: Jetson + RealSense Time: ~2 h Doc: docs/skills/FACE_AUTH.md

Recognises an enrolled administrator and unlocks restricted data for a limited session. Because a face alone is a *public* credential — anyone can photograph it — this skill requires **four independent factors**, and refuses if any one fails:

Factor	What it proves, and how
1. Identity	ArcFace embedding (<code>w600k_mbf.onnx</code> , InsightFace <code>buffalo_s</code>) through <code>onnxruntime</code> ; 512-d L2-normalised vector compared against the enrolled gallery.
2. Passive liveness	The depth camera: a real face is convex , a photo is flat . Measured as RMS residual from a best-fit plane, plus depth spread.
3. Active liveness	A randomly chosen head-turn challenge that a static image cannot satisfy.
4. Knowledge	A PIN, stored only as a salted PBKDF2-SHA256 hash — never in plaintext, not even on the robot.

1 Fetch the recognition model

JETSON

```
mkdir -p ~/face_auth/models && cd ~/face_auth/models
# w600k_mbf.onnx from the InsightFace buffalo_s pack (MobileFaceNet,
# input [None,3,112,112] -> 512-d embedding). Face detection reuses
# the CP5 model at ~/robot_ws/models/blaze_face_short_range.tflite
python3 -c "import onnxruntime; print(onnxruntime.get_available_providers())"
```

Note: recognition runs on **CPU** via onnxruntime and is fast enough (tens of milliseconds per face). This is deliberate — it adds no GPU contention and, critically, no new pip packages beyond what CP5 and CP6 already installed.

2 Enrol an administrator

JETSON

```
cd ~/face_auth
python3 face_auth.py enroll --name Sam --count 25 --timeout 60
# look at the camera; turn slightly L/R/up/down between captures
# writes gallery/sam.npz (embeddings) + sam_montage.png (visual check)
```

Note: vary your pose between shots. 25 near-identical frames produce a gallery that only recognises you sitting exactly where you enrolled.

3 Set the PIN, then authenticate

JETSON

```
python3 auth_gate.py set-pin --name Sam --pin 4729
python3 auth_gate.py authenticate
# Phase 1 look at the camera (identity + passive liveness)
# Phase 2 "look LEFT, then RIGHT" (active liveness challenge)
# Phase 3 PIN prompt (getpass, not echoed)
# -> ACCESS GRANTED, 15-minute session token
python3 auth_gate.py check-session # for the protected command to call
python3 auth_gate.py logout
```

Note: `admins.json` and `session.json` are written `0600` and live outside the repo. The gallery, the models and the session are all git-ignored — **never commit biometric data**. Audit your staged files before every commit on this skill.

TRAP AVOIDED — THE ONE THAT ACTUALLY BIT US

A printed photograph beat the first version. The original liveness test measured only the *depth range* across the face, reasoning that a flat photo has none. It does — hold a photo at a slight angle and the range across the tilted sheet is large enough to pass. Worse, the decision logic voted only over frames that had passed liveness, so the failures were discarded rather than counted. A photo of the enrolled admin authenticated successfully. **Two changes fixed it:** (1) fit a plane to the face depth and require the **RMS residual ≥ 4.5 mm** — a real face is convex and deviates from any plane, a sheet does not, at any angle; (2) **count the failures** — reject outright if more than **30 %** of frames fail liveness, instead of quietly ignoring them.

FROM THE FIELD

Measured thresholds and results. Valid-depth fraction ≥ 0.35 , depth spread ≥ 18 mm, planarity residual ≥ 4.5 mm, spoof-frame rejection above 30 %, head-turn swing ≥ 0.25 normalised nose offset. On the reference build the live administrator matched at **0.789 on 59 of 63 frames with 0 % spoof frames**; a printed photo of the same person produced **42-54 % spoof frames and was rejected** — *even when the correct PIN was supplied*. The head-turn threshold started at 0.32 and false-rejected a genuine turn measured at 0.312; it was lowered to 0.25, still far above the ~ 0 a static image produces.

GATE - proceed only when:

- ☐ The enrolled administrator authenticates and receives a session token.
- ☐ A **photograph** of that administrator is rejected **even with the correct PIN**.
- ☐ A wrong PIN is refused, and no plaintext PIN appears anywhere in `admins.json`.
- ☐ The session expires on its own and `logout` clears it.
- ☐ `git status` shows no gallery, model, or session file staged.

DECISION — BE HONEST ABOUT WHAT THIS DOES NOT STOP

This is a **convenience gate for a school administration corridor**, not a bank vault. It defeats a printed photo and a casual impostor. It has **not** been tested against a video replay on a screen, a 3D mask, or an identical twin; and the match threshold has **not** yet been calibrated against a second live person — that test needs a willing volunteer and should be run before any genuinely sensitive data sits behind it. Never make the face the *only* factor: the PIN is what makes a successful spoof insufficient on its own.

SKILL 2

Video message to the administration

Runs on: Jetson + RealSense + ReSpeaker Time: ~2 h Doc: docs/skills/VIDEO_MESSAGE.md

The robot stands in the administration and offers a visitor to record a short video message for a named administrator. The design rule is the one that governs the whole robot: **it operates entirely locally**. Recording never touches the network. Sending is a separate, deliberate **gate** that is **closed by default** — so the robot is useful with no internet at all, and only reaches outside when an administrator has explicitly opened it.

1 Install ffmpeg (the only new system package)

JETSON

```
sudo apt-get install -y ffmpeg
mkdir -p ~/video_messages && cd ~/video_messages
cp ~/robot_ws/tools/video_messenger/{video_message.py,deliver.py} .
```

2 Offer, then record

JETSON

```
python3 video_message.py offer          # the spoken invitation
python3 video_message.py record --seconds 15
# 3-2-1 countdown, then records; queues an .mp4 in outbox/
python3 video_message.py list
```

Note: `offer` also publishes the invitation on `/robot_speech`, so it becomes *audible* the moment the Pi's text-to-speech exists at CP4 — no code change needed.

TRAP AVOIDED

Two names for one microphone. ALSA calls the array `ArrayUAC10`; the Python `sounddevice` layer calls it *"ReSpeaker 4 Mic Array (UAC1.0)"*. Match on **ReSpeaker** — a matcher written against the ALSA name finds nothing and the recorder aborts before the camera even starts. And expect the same ALSA `-9985` release race as CP6: retry the open 4 times, 3 seconds apart.

TRAP AVOIDED

Audio and video drift apart if you trust the nominal frame rate. The camera advertises 30 FPS; a node doing real work receives ~24.5. Encode at 30 and the video finishes before the audio, so the speaker's voice runs progressively out of sync. The recorder therefore **counts the frames it actually captured, divides by the measured elapsed time**, and hands *that* rate to ffmpeg.

3 Inspect the gate (it is closed)

JETSON

```
python3 deliver.py init      # writes a DISABLED config template, 0600
python3 deliver.py status    # gate: CLOSED | network | queue depth
python3 deliver.py send      # refuses, exit 1 – nothing leaves the robot
```

4 Open the gate — choose a transport

Edit `~/video_messages/config.json` (mode `0600`, outside the repo, git-ignored) and set `"enabled": true` plus one transport:

Transport	What it needs
email simplest; validated	An SMTP account and an app password (never the real account password). Set <code>user</code> , <code>app_password</code> , <code>to</code> .
whatsapp official Cloud API	A free Meta developer app with the WhatsApp product: <code>access_token</code> , <code>phone_number_id</code> , and the destination number in international format.

JETSON

```
python3 deliver.py send --dry-run # shows what would go; sends nothing
python3 deliver.py send           # delivers, then moves the file to sent/
```

DECISION — WHY NOT AN UNOFFICIAL WHATSAPP LIBRARY

Several Python packages drive WhatsApp by automating a logged-in web session. They are easier to set up than the Cloud API and they are **rejected here on purpose**: they violate WhatsApp's terms of service and the realistic penalty is a **permanent ban of the school's phone number**. A messaging convenience is not worth that. Use the official API, or use email.

TRAP AVOIDED

Gmail will reject your password — that is not a typo. Google no longer accepts ordinary account passwords over SMTP; you get 535 5.7.8 BadCredentials no matter how carefully you retype it. You must enable 2-Step Verification and mint a 16-character **App Password**. Two further surprises: a **brand-new sending account is held for roughly half an hour** before its first message is delivered (it shows in "Sent" and simply has not arrived yet — do not debug a working system), and the reachability probe must use **HTTPS on port 443**, because the school network blocks the classic 8.8.8.8:53 check and makes an online robot look offline.

FROM THE FIELD

Validated end to end. A 10-second message recorded as 245 frames at a measured 24.5 FPS, muxed to H.264 + AAC with `+faststart` (0.6 MB, audio peaking at -8 dB, confirmed with `ffprobe` and a extracted thumbnail). With the gate closed, `send` exited 1 and nothing left the board. With the gate opened and an app password in place, the message was accepted by SMTP and **arrived in the administrator's inbox**, and the file moved to `sent/`. Delivery failures leave the message queued — a dropped Wi-Fi link loses nothing.

GATE - proceed only when:

- ☐ A recorded message plays back with **lip-synced audio** — this is what proves the measured-FPS fix.
- ☐ `ffprobe` reports an `h264` video stream and an `aac` audio stream.
- ☐ With `"enabled": false`, `send` exits 1 and the outbox is untouched.
- ☐ With the gate open, a message is delivered and moves to `sent/`.
- ☐ No credential and no recording is staged in git — both paths are git-ignored.

Volume I and Volume II together cover everything the reference build has actually proved. This page is the honest remainder, so nobody follows the tutorial expecting a capability that does not exist yet.

THE CRITICAL GAP - THE RASPBERRY PI 5

The robot cannot speak, and cannot move. Both live on the Pi 5 (CP2), which is not yet stood up on the reference build. Until it is, CP4 (the DDS mesh), CP7-CP9, text-to-speech, SLAM and navigation are all blocked. Nothing on the Jetson is waiting on redesign — the answers are already published on `/robot_speech` and the video-message invitation with them; they simply have no voice to come out of. Getting the Pi running is **the single highest-value next step** in the whole project.

On the reference build the Pi is blocked by four ordinary hardware problems, all of which are worth solving before the next session rather than during it:

Blocker	What is needed
SSH credentials are wrong	The image's login does not work, so the card must be re-flashed. Set the username and password during imaging and record them immediately.
No SD card reader	A USB SD reader for the workstation — without it the card cannot be re-imaged at all.
Runs very hot / unstable	A heatsink and fan. A Pi 5 under sustained ROS load throttles and drops off the network, which looks exactly like a software fault.
Power supply	A genuine 5 V / 5 A USB-C supply. Under-powering a Pi 5 causes random resets that are commonly misdiagnosed for days.

Smaller open items

- Impostor calibration.** The face-auth match threshold has only ever been tested against the enrolled administrator and a photograph of him. It needs one session with a second live person to confirm a stranger is refused.
- Voice-triggered video messages.** The recorder is driven from the command line today. Hooking a "yes" intent on `/speech/text` to `record` makes the skill conversational.
- WhatsApp transport.** Written and ready; needs a Meta developer app before it can be enabled.
- Volume I corrections.** Two values in Volume I differ from the reference build and should be reconciled on the next revision: the UDP buffer sizes (Volume I shows 4194304 / 1048576, the build uses 2147483647), and the CP1 inline pin list, which omits `protobuf==4.25.8`. CP0 is documented but was not performed on the reference build — it becomes mandatory at CP4, when the two boards must find each other on fixed addresses.

FROM THE FIELD — HOUSEKEEPING THAT PAYS FOR ITSELF

Secrets. Every credential this build uses lives in a `0600` file outside the git repository — the mail app password in `~/video_messages/config.json`, the administrator PIN as a hash in `~/face_auth/admins.json`, the GitHub token in `~/config/gh/hosts.yml`. Keep it that way, and audit the staged file list before every commit rather than trusting `.gitignore` alone.

Recordings are personal data. The `sent/` folder accumulates videos of real visitors. Decide a retention period and delete on it.

Default passwords. A board that is about to live on a school network should not still be using the password it was imaged with.